

Name :- _Pratiksha Madhav Tonde

Clg Name :- T.B.Girwalkr Polytechnic , Ambajogai

Domain :- AI - ML

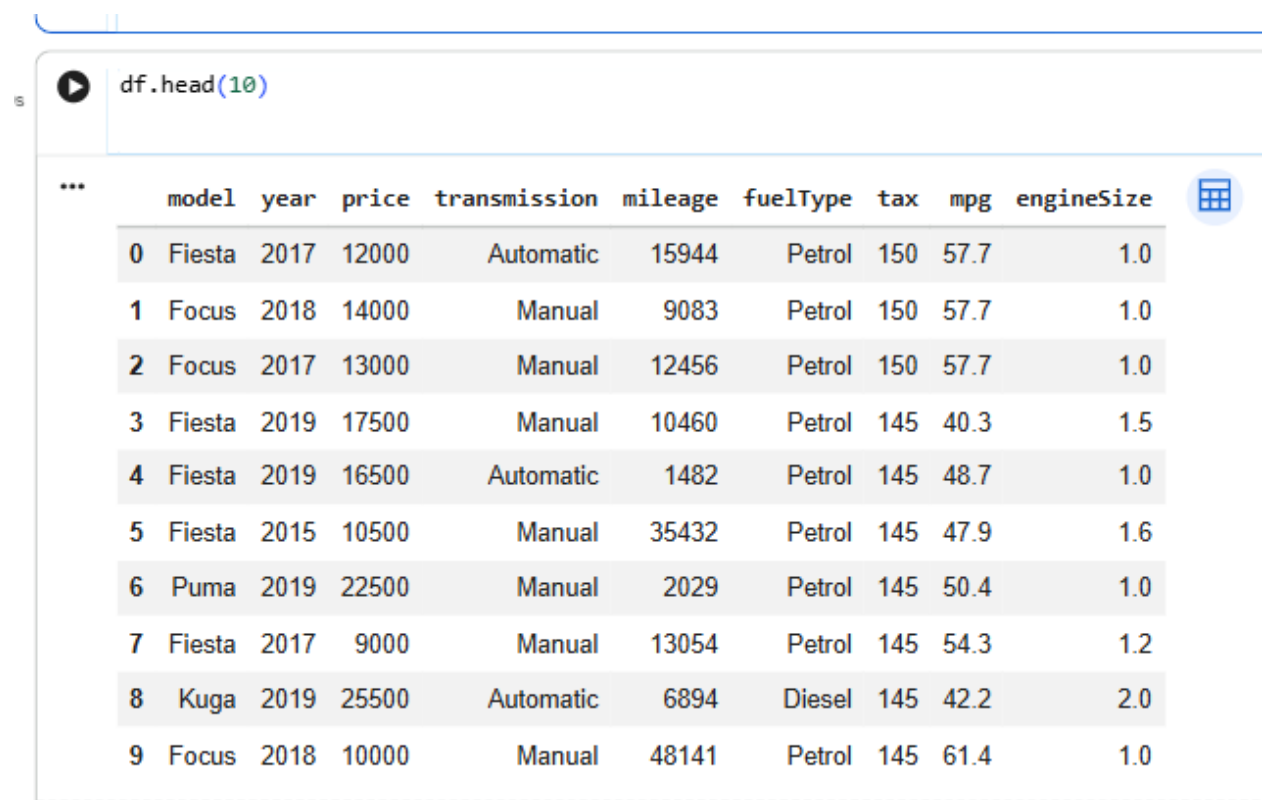
Branch Name :- Computer Engineering

1) Load the Ford Car Dataset using pandas.

Display the first 10 rows and last 5 rows.

Check the shape (number of rows and columns) and data types of all columns.

Write observations about the dataset structure in comments.



The screenshot shows a Jupyter Notebook interface. In the code cell, the command `df.head(10)` has been executed. The output is a table displaying the first 10 rows of the dataset. The columns are: model, year, price, transmission, mileage, fuelType, tax, mpg, and engineSize. The data includes various Ford models like Fiesta, Focus, and Puma, with details on their year, price, transmission type, mileage, fuel type, tax, miles per gallon (mpg), and engine size.

	model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
0	Fiesta	2017	12000	Automatic	15944	Petrol	150	57.7	1.0
1	Focus	2018	14000	Manual	9083	Petrol	150	57.7	1.0
2	Focus	2017	13000	Manual	12456	Petrol	150	57.7	1.0
3	Fiesta	2019	17500	Manual	10460	Petrol	145	40.3	1.5
4	Fiesta	2019	16500	Automatic	1482	Petrol	145	48.7	1.0
5	Fiesta	2015	10500	Manual	35432	Petrol	145	47.9	1.6
6	Puma	2019	22500	Manual	2029	Petrol	145	50.4	1.0
7	Fiesta	2017	9000	Manual	13054	Petrol	145	54.3	1.2
8	Kuga	2019	25500	Automatic	6894	Diesel	145	42.2	2.0
9	Focus	2018	10000	Manual	48141	Petrol	145	61.4	1.0

 `df.tail()`

...

	model	year	price	transmission	mileage	fuelType	tax	mpg	engineSize
17961	B-MAX	2017	8999	Manual	16700	Petrol	150	47.1	1.4
17962	B-MAX	2014	7499	Manual	40700	Petrol	30	57.7	1.0
17963	Focus	2015	9999	Manual	7010	Diesel	20	67.3	1.6
17964	KA	2018	8299	Manual	5007	Petrol	145	57.7	1.2
17965	Focus	2015	8299	Manual	5007	Petrol	22	57.7	1.0



`df.shape`

(17966, 9)

 `df.dtypes`

...

	0
model	object
year	int64
price	int64
transmission	object
mileage	int64
fuelType	object
tax	int64
mpg	float64
engineSize	float64

dtype: object

5

`df.info()`

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 17966 entries, 0 to 17965
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   model           17966 non-null  object
1   year            17966 non-null  int64
2   price           17966 non-null  int64
3   transmission    17966 non-null  object
4   mileage         17966 non-null  int64
5   fuelType        17966 non-null  object
6   tax             17966 non-null  int64
7   mpg             17966 non-null  float64
8   engineSize      17966 non-null  float64
dtypes: float64(2), int64(4), object(3)
memory usage: 1.2+ MB
```

2) Perform a detailed check on the dataset:

Find the total number of missing values in each column.

Identify and remove duplicate rows if any.

Write a summary of your findings and how you handled them in comments.

```
df.isnull().sum()
```

...	
	0
model	0
year	0
price	0
transmission	0
mileage	0
fuelType	0
tax	0
mpg	0
engineSize	0

dtype: int64

```
5] df.isnull().sum().sum()
```

```
0s
```

```
... np.int64(0)
```

```
df.duplicated().sum()
```

```
... np.int64(154)
```

- 3) Generate the statistical summary of all numeric columns using `describe()`. Identify minimum, maximum, mean, and median values for key columns like price, mileage, and year.

 `df.describe()`

...	year	price	mileage	tax	mpg	engineSize	
count	17966.000000	17966.000000	17966.000000	17966.000000	17966.000000	17966.000000	
mean	2016.866470	12279.534844	23362.608761	113.329456	57.906980	1.350807	
std	2.050336	4741.343657	19472.054349	62.012456	10.125696	0.432367	
min	1996.000000	495.000000	1.000000	0.000000	20.800000	0.000000	
25%	2016.000000	8999.000000	9987.000000	30.000000	52.300000	1.000000	
50%	2017.000000	11291.000000	18242.500000	145.000000	58.900000	1.200000	
75%	2018.000000	15299.000000	31060.000000	145.000000	65.700000	1.500000	
max	2060.000000	54995.000000	177644.000000	580.000000	201.800000	5.000000	

[20]  Os

```
df["price"].min()
df["price"].max()
df["price"].mean()
df["price"].median()
```

11291.0

[21]  Os

```
df["price"].median()
```

11291.0

[22]  Os 

```
df["price"].mean()
```

... np.float64(12279.534843593454)

[23]  Os

```
df["price"].max()
```

54995

[24]  Os 

```
df["price"].min()
```

... 495

```
df["mileage"].min()  
df["mileage"].max()  
df["mileage"].mean()  
df["mileage"].median()
```

18242.5

```
df["mileage"].min()
```

1



```
df["mileage"].max()
```

... 177644

```
df["mileage"].mean()
```

np.float64(23362.608760992985)



```
df["mileage"].median()
```

... 18242.5

```
df["year"].min()  
df["year"].max()  
df["year"].mean()  
df["year"].median()
```

2017.0

```
df["year"].min()
```

1996

```
df["year"].max()
```

2060

```
df["year"].mean()
```

np.float64(2016.8664699988867)

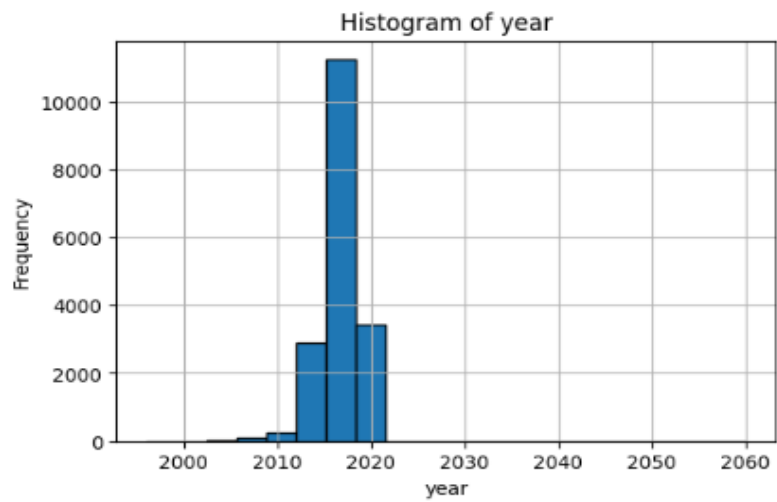
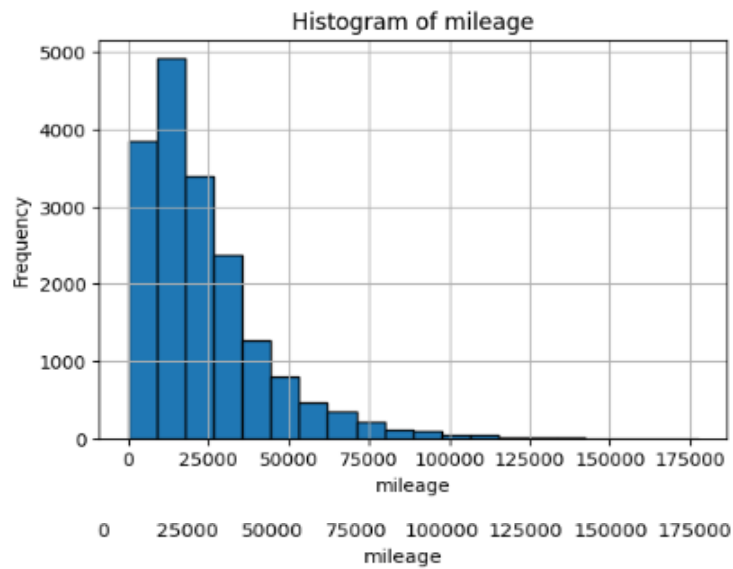


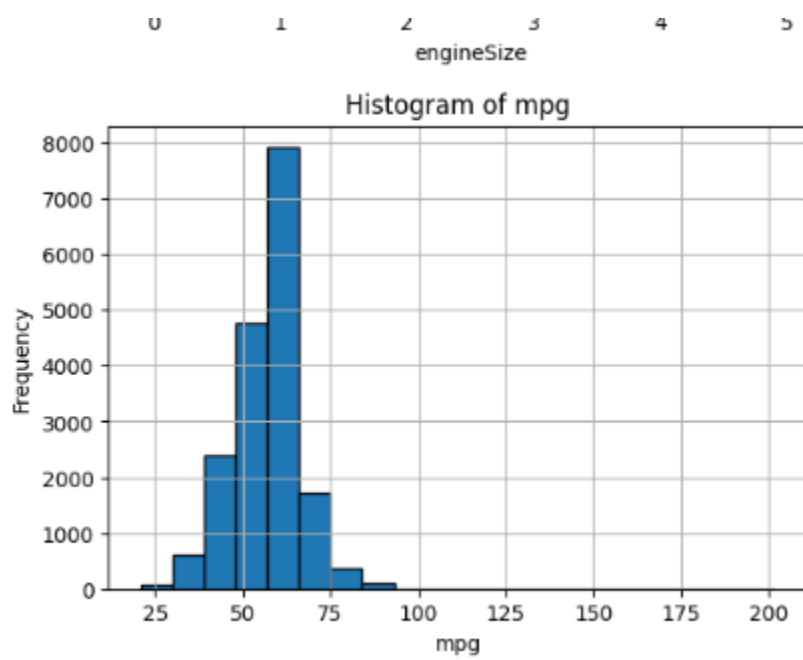
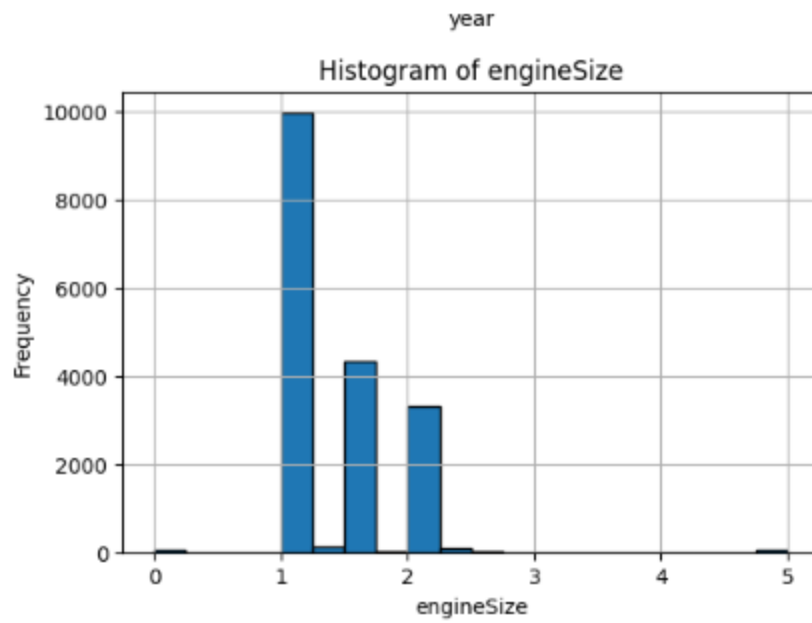
```
df["year"].median()
```

... 2017.0

4)

...





5) Clearly identify and list:

Independent Features (Input Features)

Dependent Feature (Output/Target Feature)

Justify your choice with reasoning.

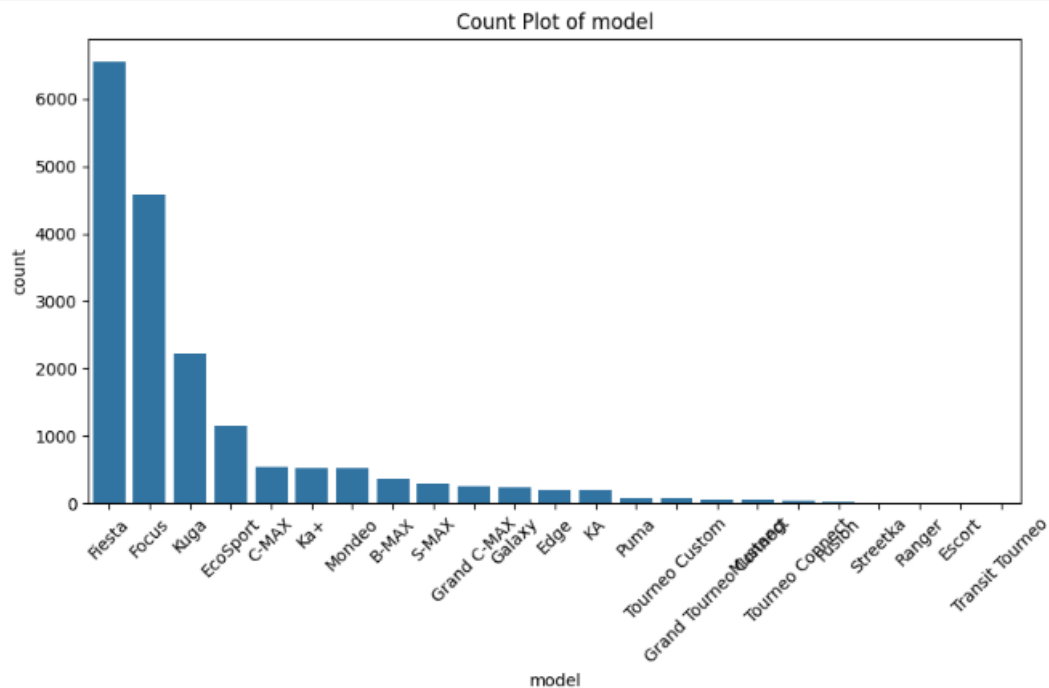
```
X = df.drop('price', axis=1)
print(X.columns)

*** Index(['model', 'year', 'transmission', 'mileage', 'fuelType', 'tax', 'mpg',
          'engineSize'],
          dtype='object')
```

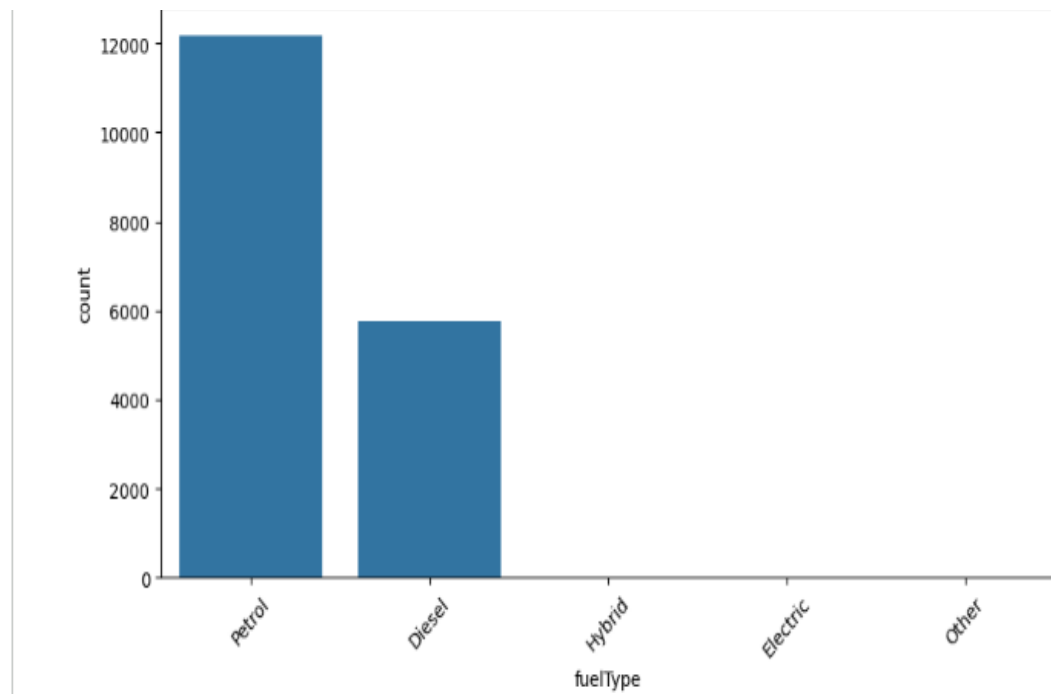
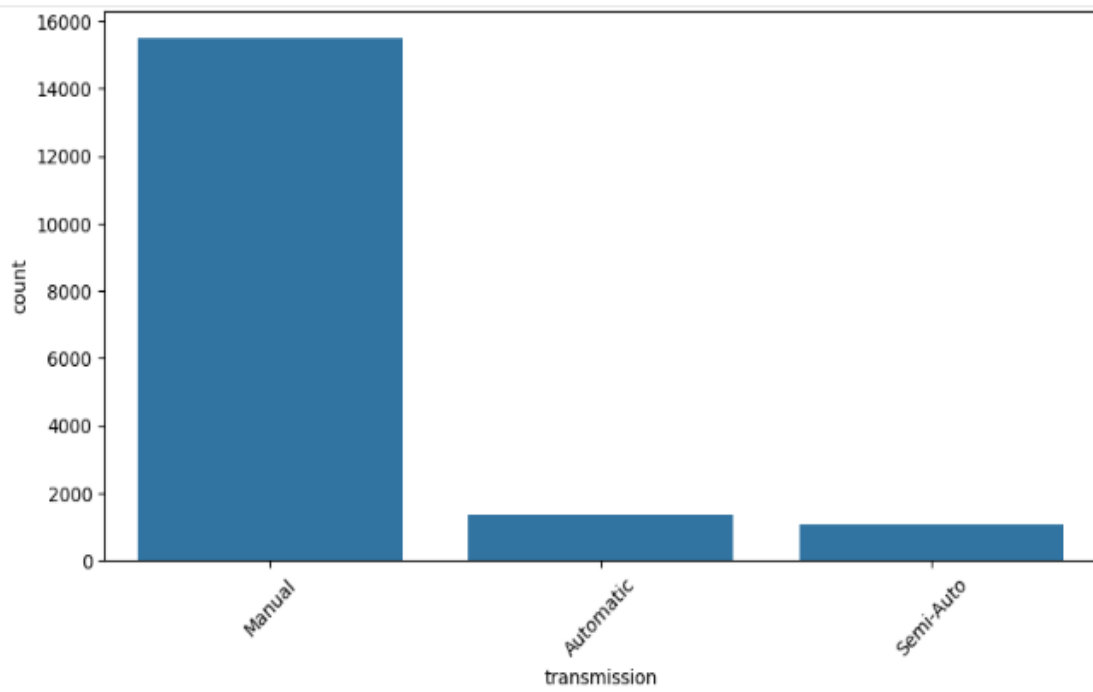
6) Create count plots for all categorical columns (fuelType, transmission, model, etc.) using seaborn.

Analyze which categories are most common.

Write insights about market trends based on these plots in comments.

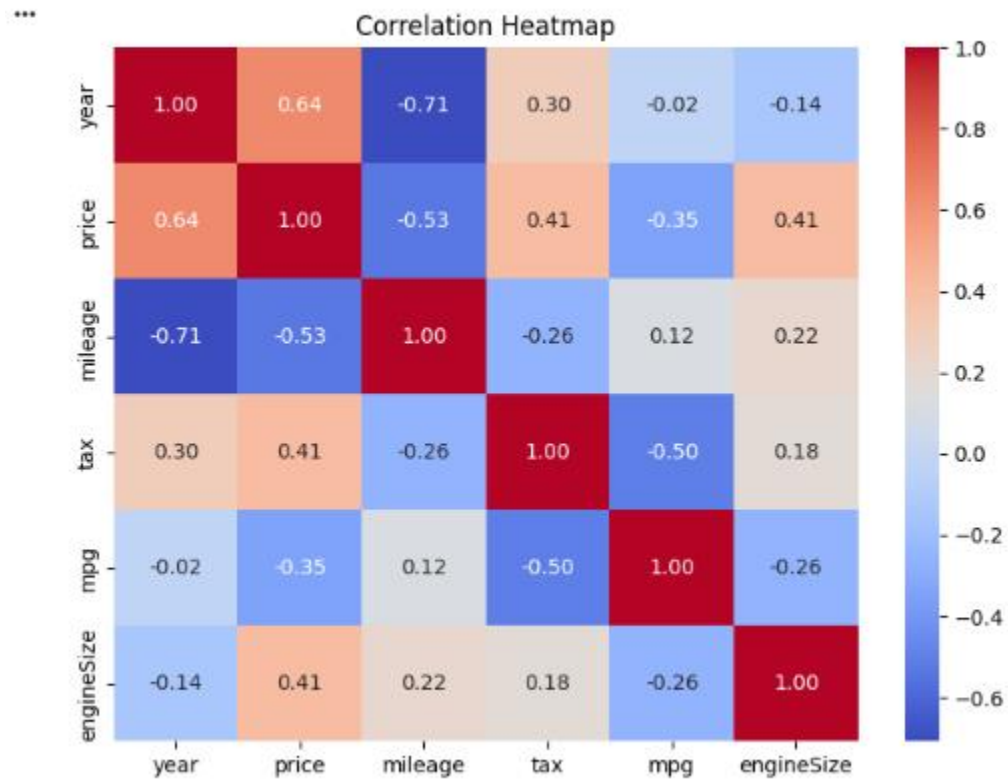


```
plt.show()
```



7)

```
plt.title("Correlation Heatmap")  
plt.show()
```



8)

```
from sklearn.preprocessing import StandardScaler
X = df.drop('price', axis=1)
X_numeric = X.select_dtypes(include=['int64', 'float64'])
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_numeric)
X_scaled = pd.DataFrame(X_scaled, columns=X_numeric.columns)
X_scaled.head()
```

```
***
```

	year	mileage	tax	mpg	engineSize
0	0.065128	-0.380998	0.591358	-0.020442	-0.811386
1	0.552866	-0.733359	0.591358	-0.020442	-0.811386
2	0.065128	-0.560132	0.591358	-0.020442	-0.811386
3	1.040605	-0.662640	0.510727	-1.738890	0.345070
4	1.040605	-1.123724	0.510727	-0.909294	-0.811386

Next steps: [Generate code with X_scaled](#) [New interactive sheet](#)

9)

